LAPORAN
UJIAN TENGAH SEMESTER

APLIKASI PENGHITUNG TAGIHAN LISTRIK

Penerapan Access Modifier (Encapsulation) dan Inheritance pada Java

Disusun oleh

NAMA : Ahmad Choiruman Qotadah

NIM : 32602500039

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

UNIVERSITAS ISLAM SULTAN AGUNG

SEMARANG

2026

1. Pendahuluan

1.1 Latar Belakang

Tagihan listrik dihitung dari banyak komponen, yaitu pemakaian energi (kWh), tarif sesuai golongan pelanggan, biaya beban, dan pajak penerangan jalan. Setiap golongan pelanggan (rumah tangga, bisnis, dan industri) memiliki tarif dan aturan yang berbeda, tetapi data dasar dan alur perhitungannya sama. Kondisi ini cocok dimodelkan dengan pemrograman berorientasi objek.

Pada Ujian Tengah Semester ini dibuat sebuah aplikasi Java berbasis konsol bernama Aplikasi Penghitung Tagihan Listrik. Aplikasi ini bukan sekadar kumpulan class, melainkan program yang berfungsi menghitung dan mencetak tagihan listrik pelanggan, dengan menerapkan dua prinsip OOP: Access Modifier / Encapsulation dan Inheritance.

1.2 Tujuan

- Membuat aplikasi Java yang memiliki fungsi nyata, yaitu menghitung tagihan listrik.
- Menerapkan access modifier (private, protected, public) dan enkapsulasi melalui getter dan setter dengan validasi.
- Menerapkan inheritance dengan satu superclass abstrak dan tiga subclass berdasarkan golongan pelanggan.

2. Deskripsi Aplikasi

2.1 Fitur

- Menambahkan pelanggan baru (golongan Rumah Tangga, Bisnis, atau Industri) beserta pembacaan meter awal dan akhir.
- Menghitung pemakaian kWh, biaya pemakaian, biaya beban, pajak penerangan jalan (PPJ 10%), dan total tagihan.
- Menampilkan daftar pelanggan yang sudah dimasukkan.
- Mencetak tagihan pelanggan berdasarkan ID.
- Validasi input, misalnya meter akhir tidak boleh lebih kecil dari meter awal.

2.2 Rumus Perhitungan

Pemakaian (kWh) = Meter Akhir - Meter Awal

Biaya Pemakaian = Pemakaian x Tarif per kWh

PPJ = (Biaya Pemakaian + Biaya Beban) x 10%

Total Tagihan = Biaya Pemakaian + Biaya Beban + PPJ

2.3 Tarif yang Digunakan

Tarif berikut adalah nilai simulasi untuk keperluan tugas dan tidak mewakili tarif resmi PLN yang berlaku.

Golongan	Tarif per kWh	Biaya Beban	Keterangan
Rumah Tangga	Rp1.444,70	Rp15.000 / bulan	Tetap
Bisnis	Rp1.699,53	Rp50.000 / bulan	Memiliki atribut nama usaha
Industri	Rp1.114,74	Rp35.000 x daya (kVA)	Bergantung pada daya terpasang

3. Penerapan Prinsip OOP

3.1 Access Modifier / Encapsulation

Enkapsulasi adalah pembungkusan data dan method dalam satu class serta pembatasan akses langsung ke data tersebut. Pada aplikasi ini, enkapsulasi diterapkan dengan cara berikut.

Modifier	Penerapan pada kode	Alasan
private	Atribut idPelanggan, nama, alamat, dayaVA, meterAwal, meterAkhir pada class Pelanggan; atribut namaUsaha pada PelangganBisnis; konstanta tarif pada setiap subclass.	Data tidak boleh diubah langsung dari luar class agar tidak diisi nilai yang salah.
public	Constructor, getter, setter, hitungTotalTagihan(), dan cetakTagihan().	Merupakan pintu masuk resmi yang boleh dipakai class lain, termasuk Main.
protected	getTarifPerKwh(), getBiayaBeban(), hitungPajak(), dan infoTambahan().	Hanya dibutuhkan oleh class turunan dan package yang sama, tidak perlu terbuka untuk umum.

Setter tidak sekadar menyimpan nilai, tetapi memvalidasinya terlebih dahulu. Contohnya setNama() menolak nama kosong, setDayaVA() menolak daya nol atau negatif, dan setPembacaanMeter() menolak meter akhir yang lebih kecil dari meter awal. Jika data tidak valid, method melempar IllegalArgumentException yang ditangkap di class Main dan ditampilkan sebagai pesan kepada pengguna.

3.2 Inheritance

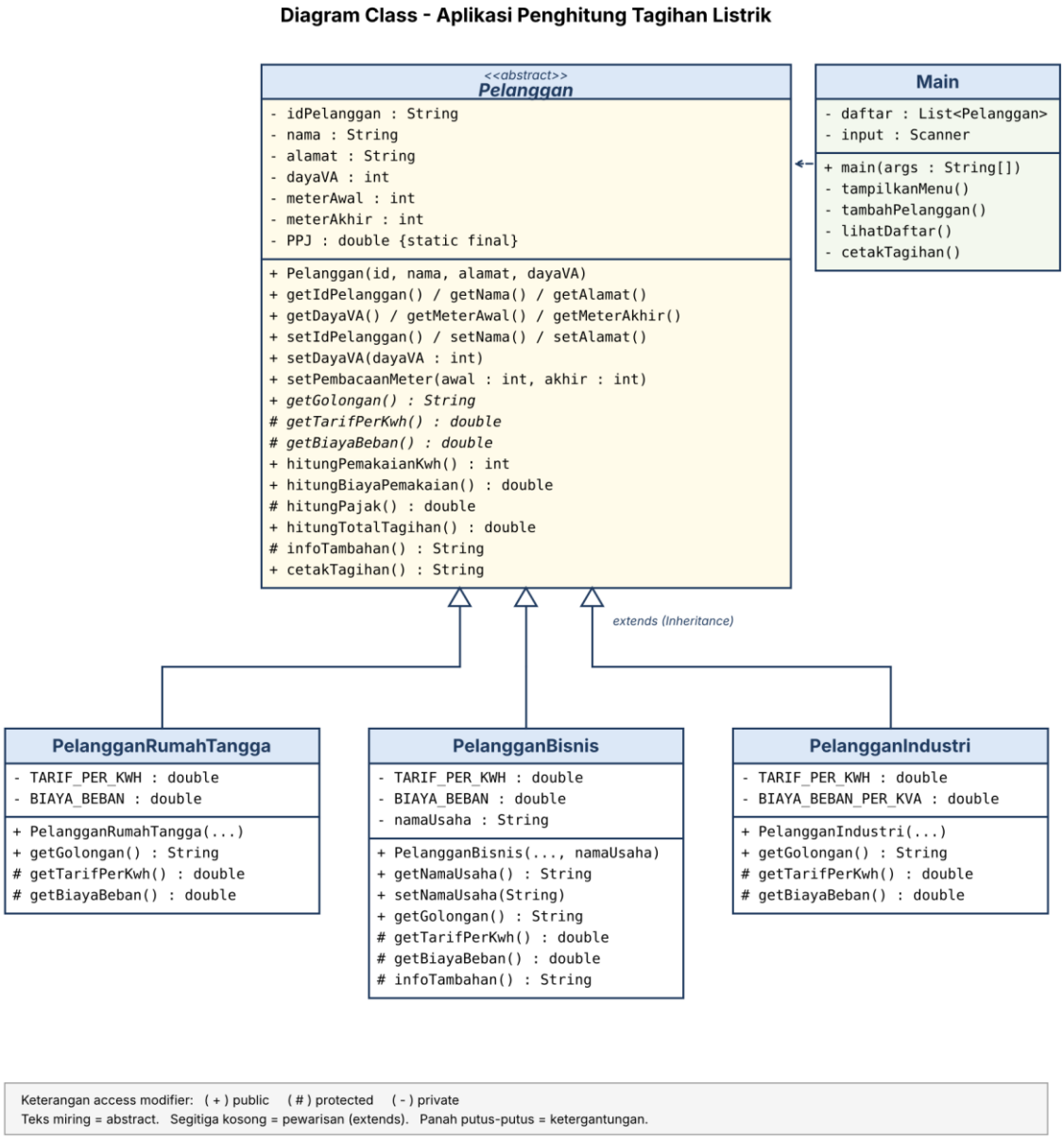
Pewarisan digunakan agar atribut dan perilaku yang sama tidak ditulis berulang pada setiap golongan pelanggan. Class Pelanggan menjadi superclass abstrak yang berisi data pelanggan, validasi, dan alur perhitungan tagihan. Tiga subclass mewarisinya dengan kata kunci extends dan hanya menuliskan hal yang berbeda.

- **PelangganRumahTangga** mengisi tarif dan biaya beban tetap untuk golongan rumah tangga.
- **PelangganBisnis** mengisi tarif bisnis, menambah atribut namaUsaha beserta getter dan setter, dan menimpa infoTambahan() agar nama usaha tampil pada tagihan.
- **PelangganIndustri** mengisi tarif industri dan menghitung biaya beban dari daya terpasang (kVA).

Method getGolongan(), getTarifPerKwh(), dan getBiayaBeban() dideklarasikan abstract di superclass sehingga setiap subclass wajib mengimplementasikannya dengan @Override. Pada class Main, semua objek disimpan dalam satu List<Pelanggan> dan dipanggil dengan cara yang sama, tetapi hasil hitungnya menyesuaikan jenis objeknya (polymorphism).

4. Diagram Class

Diagram berikut menunjukkan hubungan antar class, atribut, method, dan access modifier yang digunakan.



Gambar 1. Diagram class Aplikasi Penghitung Tagihan Listrik

5. Kode Program

Seluruh file berada pada package tagihanlistrik di dalam folder src.

5.1 Pelanggan.java (Superclass Abstrak)

```
package tagihanlistrik;

import java.text.NumberFormat;
import java.util.Locale;

/**
 * Superclass abstrak untuk semua jenis pelanggan listrik.
 *
 * Prinsip yang diterapkan:
 *  1. Access Modifier / Encapsulation
 *     - atribut dibuat private, hanya bisa diakses lewat getter/setter
 *     - setter memvalidasi data sebelum disimpan
 *     - method yang hanya dipakai subclass dibuat protected
 *  2. Inheritance
 *     - PelangganRumahTangga, PelangganBisnis, dan PelangganIndustri
 *       mewarisi class ini dengan "extends"
 */
public abstract class Pelanggan {

    // Konstanta tarif pajak penerangan jalan (PPJ) 10%
    private static final double PPJ = 0.10;

    // ===== Atribut private (Encapsulation) =====
    private String idPelanggan;
    private String nama;
    private String alamat;
    private int dayaVA;
    private int meterAwal;
    private int meterAkhir;

    // ===== Constructor =====
    public Pelanggan(String idPelanggan, String nama, String alamat, int dayaVA)
{
        setIdPelanggan(idPelanggan);
        setNama(nama);
        setAlamat(alamat);
        setDayaVA(dayaVA);
    }

    // ===== Getter =====
    public String getIdPelanggan() {
        return idPelanggan;
    }

    public String getNama() {
        return nama;
    }

    public String getAlamat() {
        return alamat;
    }

    public int getDayaVA() {
        return dayaVA;
    }

    public int getMeterAwal() {
        return meterAwal;
    }
```

```
    public int getMeterAkhir() {
        return meterAkhir;
    }

    // ===== Setter dengan validasi =====
    public void setIdPelanggan(String idPelanggan) {
        if (idPelanggan == null || idPelanggan.trim().isEmpty()) {
            throw new IllegalArgumentException("ID pelanggan tidak boleh
kosong.");
        }
        this.idPelanggan = idPelanggan.trim();
    }

    public void setNama(String nama) {
        if (nama == null || nama.trim().isEmpty()) {
            throw new IllegalArgumentException("Nama tidak boleh kosong.");
        }
        this.nama = nama.trim();
    }

    public void setAlamat(String alamat) {
        if (alamat == null || alamat.trim().isEmpty()) {
            throw new IllegalArgumentException("Alamat tidak boleh kosong.");
        }
        this.alamat = alamat.trim();
    }

    public void setDayaVA(int dayaVA) {
        if (dayaVA <= 0) {
            throw new IllegalArgumentException("Daya harus lebih dari 0 VA.");
        }
        this.dayaVA = dayaVA;
    }

    /**
     * Mengisi pembacaan meter bulan ini.
     * Meter akhir tidak boleh lebih kecil dari meter awal.
     */
    public void setPembacaanMeter(int meterAwal, int meterAkhir) {
        if (meterAwal < 0 || meterAkhir < 0) {
            throw new IllegalArgumentException("Angka meter tidak boleh
negatif.");
        }
        if (meterAkhir < meterAwal) {
            throw new IllegalArgumentException("Meter akhir tidak boleh lebih
kecil dari meter awal.");
        }
        this.meterAwal = meterAwal;
        this.meterAkhir = meterAkhir;
    }

    // ===== Method abstrak: wajib di-override oleh subclass =====

    /** Nama golongan tarif, contoh: "Rumah Tangga". */
    public abstract String getGolongan();

    /** Tarif per kWh (Rp) sesuai golongan. */
    protected abstract double getTarifPerKwh();

    /** Biaya beban tetap per bulan (Rp) sesuai golongan. */
    protected abstract double getBiayaBeban();

    // ===== Perhitungan tagihan =====

    public int hitungPemakaianKwh() {
        return meterAkhir - meterAwal;
    }
```

```
    public double hitungBiayaPemakaian() {
        return hitungPemakaianKwh() * getTarifPerKwh();
    }

    // protected: hanya perlu dipanggil di dalam class turunan / package
    protected double hitungPajak() {
        return (hitungBiayaPemakaian() + getBiayaBeban()) * PPJ;
    }

    public double hitungTotalTagihan() {
        return hitungBiayaPemakaian() + getBiayaBeban() + hitungPajak();
    }

    // ===== Utilitas tampilan =====

    protected static String rupiah(double nilai) {
        NumberFormat nf =
NumberFormat.getCurrencyInstance(Locale.forLanguageTag("id-ID"));
        nf.setMaximumFractionDigits(0);
        return nf.format(nilai);
    }

    // Tarif ditampilkan dengan 2 desimal agar sesuai tarif aslinya
    protected static String rupiahDesimal(double nilai) {
        NumberFormat nf =
NumberFormat.getCurrencyInstance(Locale.forLanguageTag("id-ID"));
        nf.setMinimumFractionDigits(2);
        nf.setMaximumFractionDigits(2);
        return nf.format(nilai);
    }

    /** Hook: subclass boleh menambah baris informasi (default kosong). */
    protected String infoTambahan() {
        return "";
    }

    public String cetakTagihan() {
        StringBuilder sb = new StringBuilder();
        sb.append("========================================\n");
        sb.append("           TAGIHAN LISTRIK BULANAN        \n");
        sb.append("========================================\n");
        sb.append(String.format("ID Pelanggan   : %s%n", idPelanggan));
        sb.append(String.format("Nama           : %s%n", nama));
        sb.append(String.format("Alamat         : %s%n", alamat));
        sb.append(String.format("Golongan       : %s (%d VA)%n", getGolongan(),
dayaVA));
        sb.append(infoTambahan());
        sb.append("----------------------------------------\n");
        sb.append(String.format("Meter Awal     : %d%n", meterAwal));
        sb.append(String.format("Meter Akhir    : %d%n", meterAkhir));
        sb.append(String.format("Pemakaian      : %d kWh%n",
hitungPemakaianKwh()));
        sb.append(String.format("Tarif / kWh    : %s%n",
rupiahDesimal(getTarifPerKwh())));
        sb.append("----------------------------------------\n");
        sb.append(String.format("Biaya Pemakaian: %s%n",
rupiah(hitungBiayaPemakaian())));
        sb.append(String.format("Biaya Beban    : %s%n",
rupiah(getBiayaBeban())));
        sb.append(String.format("PPJ (10%%)      : %s%n",
rupiah(hitungPajak())));
        sb.append("========================================\n");
        sb.append(String.format("TOTAL TAGIHAN  : %s%n",
rupiah(hitungTotalTagihan())));
        sb.append("========================================\n");
        return sb.toString();
    }
}
```

5.2 PelangganRumahTangga.java

```
package tagihanlistrik;

/**
 * Subclass: pelanggan golongan Rumah Tangga.
 * Mewarisi seluruh atribut dan perilaku dari Pelanggan (Inheritance).
 */
public class PelangganRumahTangga extends Pelanggan {

    private static final double TARIF_PER_KWH = 1444.70;
    private static final double BIAYA_BEBAN = 15000;

    public PelangganRumahTangga(String idPelanggan, String nama, String alamat, int
dayaVA) {
        super(idPelanggan, nama, alamat, dayaVA);
    }

    @Override
    public String getGolongan() {
        return "Rumah Tangga";
    }

    @Override
    protected double getTarifPerKwh() {
        return TARIF_PER_KWH;
    }

    @Override
    protected double getBiayaBeban() {
        return BIAYA_BEBAN;
    }
}
```

5.3 PelangganBisnis.java

```
package tagihanlistrik;

/**
 * Subclass: pelanggan golongan Bisnis (toko, kantor, warung).
 * Selain mewarisi Pelanggan, class ini menambah atribut sendiri: namaUsaha.
 */
public class PelangganBisnis extends Pelanggan {

    private static final double TARIF_PER_KWH = 1699.53;
    private static final double BIAYA_BEBAN = 50000;

    private String namaUsaha;

    public PelangganBisnis(String idPelanggan, String nama, String alamat, int dayaVA,
String namaUsaha) {
        super(idPelanggan, nama, alamat, dayaVA);
        setNamaUsaha(namaUsaha);
    }

    public String getNamaUsaha() {
        return namaUsaha;
    }

    public void setNamaUsaha(String namaUsaha) {
        if (namaUsaha == null || namaUsaha.trim().isEmpty()) {
            throw new IllegalArgumentException("Nama usaha tidak boleh kosong.");
        }
```

```
        this.namaUsaha = namaUsaha.trim();
    }

    @Override
    public String getGolongan() {
        return "Bisnis";
    }

    @Override
    protected double getTarifPerKwh() {
        return TARIF_PER_KWH;
    }

    @Override
    protected double getBiayaBeban() {
        return BIAYA_BEBAN;
    }

    // Override hook untuk menambahkan informasi nama usaha pada tagihan
    @Override
    protected String infoTambahan() {
        return "Nama Usaha     : " + namaUsaha + "\n";
    }
}
```

5.4 PelangganIndustri.java

```
package tagihanlistrik;

/**
 * Subclass: pelanggan golongan Industri.
 * Memiliki aturan tambahan: biaya beban dihitung dari daya terpasang.
 */
public class PelangganIndustri extends Pelanggan {

    private static final double TARIF_PER_KWH = 1114.74;
    private static final double BIAYA_BEBAN_PER_KVA = 35000;

    public PelangganIndustri(String idPelanggan, String nama, String alamat, int dayaVA)
{
        super(idPelanggan, nama, alamat, dayaVA);
    }

    @Override
    public String getGolongan() {
        return "Industri";
    }

    @Override
    protected double getTarifPerKwh() {
        return TARIF_PER_KWH;
    }

    // Biaya beban industri = Rp35.000 x daya dalam kVA
    @Override
    protected double getBiayaBeban() {
        double kva = getDayaVA() / 1000.0;
        return BIAYA_BEBAN_PER_KVA * kva;
    }
}
```

5.5 Main.java

```
package tagihanlistrik;
```

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

/**
 * Program utama: Aplikasi Penghitung Tagihan Listrik (berbasis konsol).
 */
public class Main {

    private static final List<Pelanggan> daftar = new ArrayList<>();
    private static final Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        boolean jalan = true;
        while (jalan) {
            tampilkanMenu();
            String pilihan = input.nextLine().trim();
            switch (pilihan) {
                case "1":
                    tambahPelanggan();
                    break;
                case "2":
                    lihatDaftar();
                    break;
                case "3":
                    cetakTagihan();
                    break;
                case "0":
                    jalan = false;
                    System.out.println("Terima kasih. Program selesai.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
        }
    }

    private static void tampilkanMenu() {
        System.out.println();
        System.out.println("=== APLIKASI PENGHITUNG TAGIHAN LISTRIK ===");
        System.out.println("1. Tambah pelanggan & hitung tagihan");
        System.out.println("2. Lihat daftar pelanggan");
        System.out.println("3. Cetak tagihan pelanggan");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");
    }

    private static void tambahPelanggan() {
        try {
            System.out.println("Golongan: 1) Rumah Tangga  2) Bisnis  3) Industri");
            System.out.print("Pilih golongan: ");
            String gol = input.nextLine().trim();

            System.out.print("ID Pelanggan : ");
            String id = input.nextLine();
            System.out.print("Nama         : ");
            String nama = input.nextLine();
            System.out.print("Alamat       : ");
            String alamat = input.nextLine();
            System.out.print("Daya (VA)    : ");
            int daya = Integer.parseInt(input.nextLine().trim());

            Pelanggan p;
            switch (gol) {
                case "1":
                    p = new PelangganRumahTangga(id, nama, alamat, daya);
                    break;
                case "2":
```

```
                    System.out.print("Nama Usaha   : ");
                    String usaha = input.nextLine();
                    p = new PelangganBisnis(id, nama, alamat, daya, usaha);
                    break;
                case "3":
                    p = new PelangganIndustri(id, nama, alamat, daya);
                    break;
                default:
                    System.out.println("Golongan tidak valid.");
                    return;
            }

            System.out.print("Meter awal   : ");
            int awal = Integer.parseInt(input.nextLine().trim());
            System.out.print("Meter akhir  : ");
            int akhir = Integer.parseInt(input.nextLine().trim());
            p.setPembacaanMeter(awal, akhir);

            daftar.add(p);
            System.out.println("\nData tersimpan.\n");
            System.out.println(p.cetakTagihan()); // polymorphism
        } catch (NumberFormatException e) {
            System.out.println("Input harus berupa angka. Data dibatalkan.");
        } catch (IllegalArgumentException e) {
            System.out.println("Data tidak valid: " + e.getMessage());
        }
    }

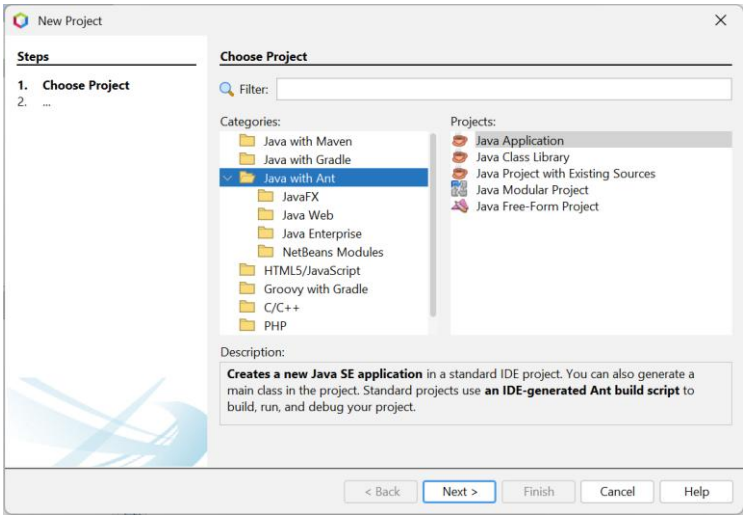
    private static void lihatDaftar() {
        if (daftar.isEmpty()) {
            System.out.println("Belum ada data pelanggan.");
            return;
        }
        System.out.println();
        System.out.printf("%-4s %-10s %-18s %-14s %12s%n", "No", "ID", "Nama",
"Golongan", "kWh");
        int no = 1;
        for (Pelanggan p : daftar) {
            System.out.printf("%-4d %-10s %-18s %-14s %12d%n",
                    no++, p.getIdPelanggan(), p.getNama(), p.getGolongan(),
p.hitungPemakaianKwh());
        }
    }

    private static void cetakTagihan() {
        System.out.print("Masukkan ID pelanggan: ");
        String id = input.nextLine().trim();
        for (Pelanggan p : daftar) {
            if (p.getIdPelanggan().equalsIgnoreCase(id)) {
                System.out.println();
                System.out.println(p.cetakTagihan());
                return;
            }
        }
        System.out.println("Pelanggan dengan ID tersebut tidak ditemukan.");
    }
}
```

6. Cara Menjalankan Program

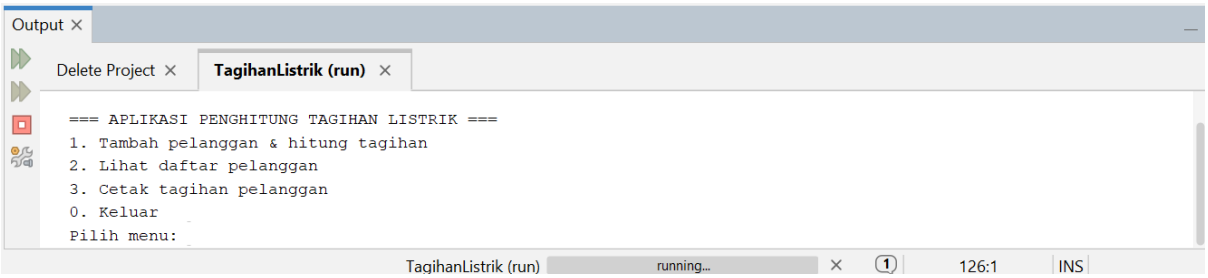
Program ini dijalankan menggunakan IDE Apache NetBeans dengan langkah-langkah berikut:

1. Buka NetBeans, pilih File > New Project > Java with Ant > Java Application. Beri nama proyek TagihanListrik, hilangkan centang Create Main Class, lalu klik Finish.



Gambar 2. Pembuatan Projek Baru menggunakan NetBeans

2. Klik kanan Source Packages > New > Java Package, beri nama tagihanlistrik.
3. Klik kanan package tersebut, pilih New > Java Class, lalu buat lima class: Pelanggan, PelangganRumahTangga, PelangganBisnis, PelangganIndustri, dan Main. Tempel kode program ke masing-masing class (kode ada di bab 5).
4. Klik kanan Main.java, lalu pilih Run File (Shift+F6).



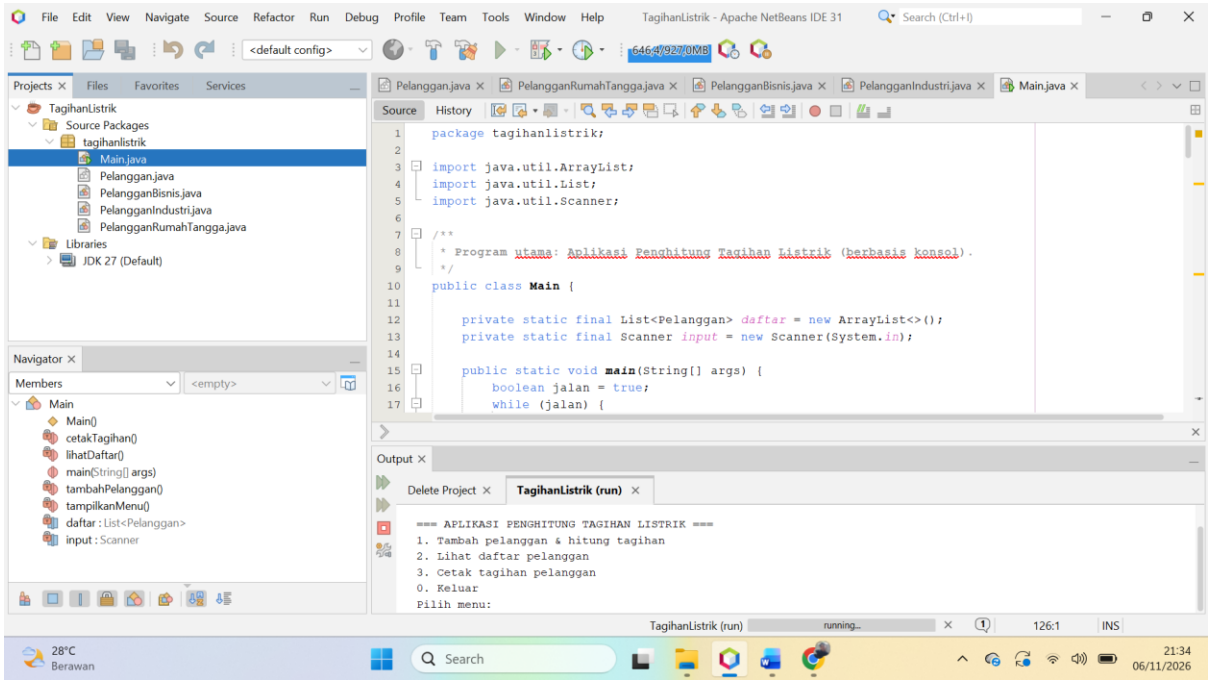
Gambar 3. Menjalankan Projek TagihanListrik menggunakan NetBeans

Setelah program berjalan, pilih menu 1 untuk menambah pelanggan, menu 2 untuk melihat daftar, menu 3 untuk mencetak tagihan, dan menu 0 untuk keluar.

7. Dokumentasi Hasil Pengujian

Bagian ini diisi dengan screenshot hasil praktik. Setiap skenario di bawah dilengkapi hasil yang diharapkan sebagai pembanding terhadap output program.

7.1 Struktur Proyek dan Kompilasi

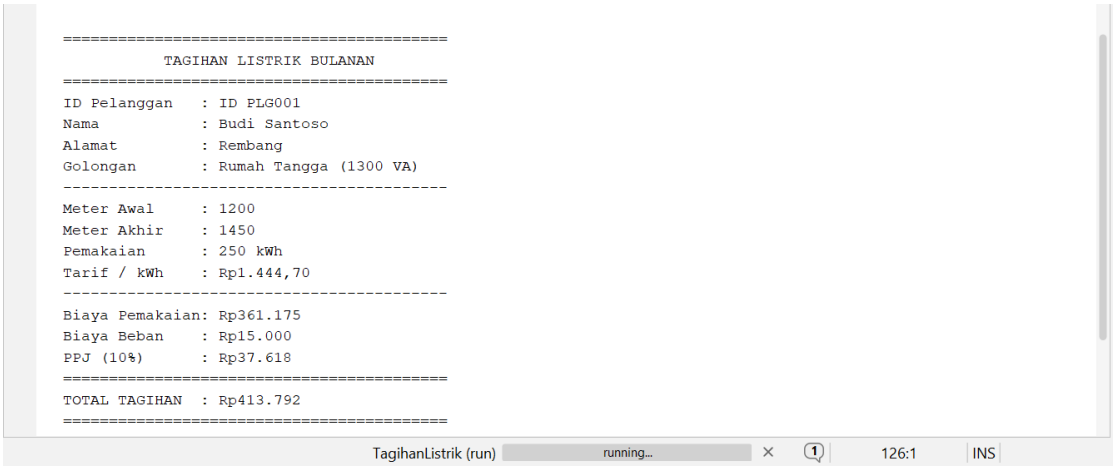


Gambar 4. Struktur proyek dan proses kompilasi

7.2 Skenario Uji

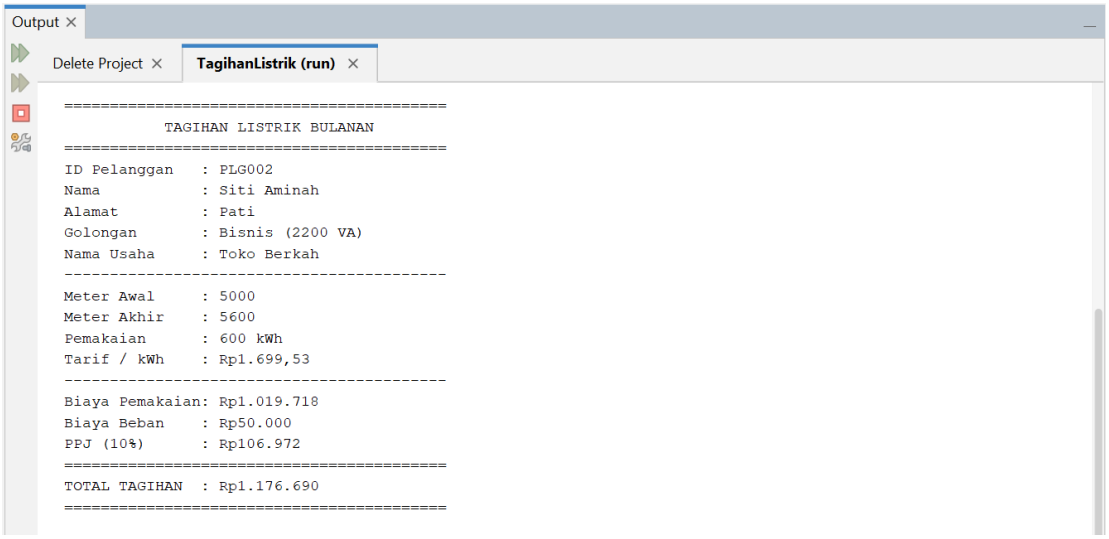
No	Skenario	Data Masukan	Hasil yang Diharapkan
1	Pelanggan Rumah Tangga	Golongan 1, ID PLG001, Budi Santoso, 1300 VA, meter 1200 ke 1450	250 kWh; total tagihan Rp413.792
2	Pelanggan Bisnis	Golongan 2, ID PLG002, Siti Aminah, 2200 VA, usaha Toko Berkah, meter 5000 ke 5600	600 kWh; total tagihan Rp1.176.690; nama usaha tampil
3	Pelanggan Industri	Golongan 3, ID PLG003, PT Maju Jaya, 41500 VA, meter 10000 ke 18000	8000 kWh; biaya beban Rp1.452.500; total tagihan Rp11.407.462
4	Validasi meter	Meter awal 500, meter akhir 400	Muncul pesan: meter akhir tidak boleh lebih kecil dari meter awal; data tidak disimpan
5	Validasi input angka	Daya diisi huruf, misalnya abc	Muncul pesan: input harus berupa angka; data dibatalkan
6	Lihat daftar pelanggan	Menu 2 setelah skenario 1 sampai 3	Tabel berisi tiga pelanggan beserta golongan dan kWh
7	Cetak tagihan berdasarkan ID	Menu 3, ID PLG002	Tagihan Siti Aminah tampil lengkap

7.3 Skenario 1: Pelanggan Rumah Tangga



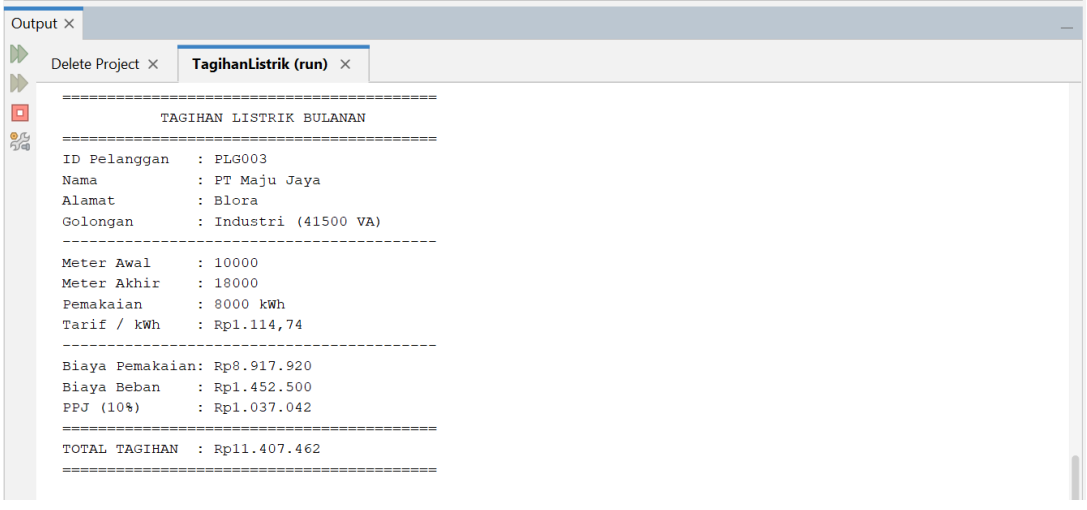
Gambar 5. Tagihan pelanggan Rumah Tangga

7.4 Skenario 2: Pelanggan Bisnis



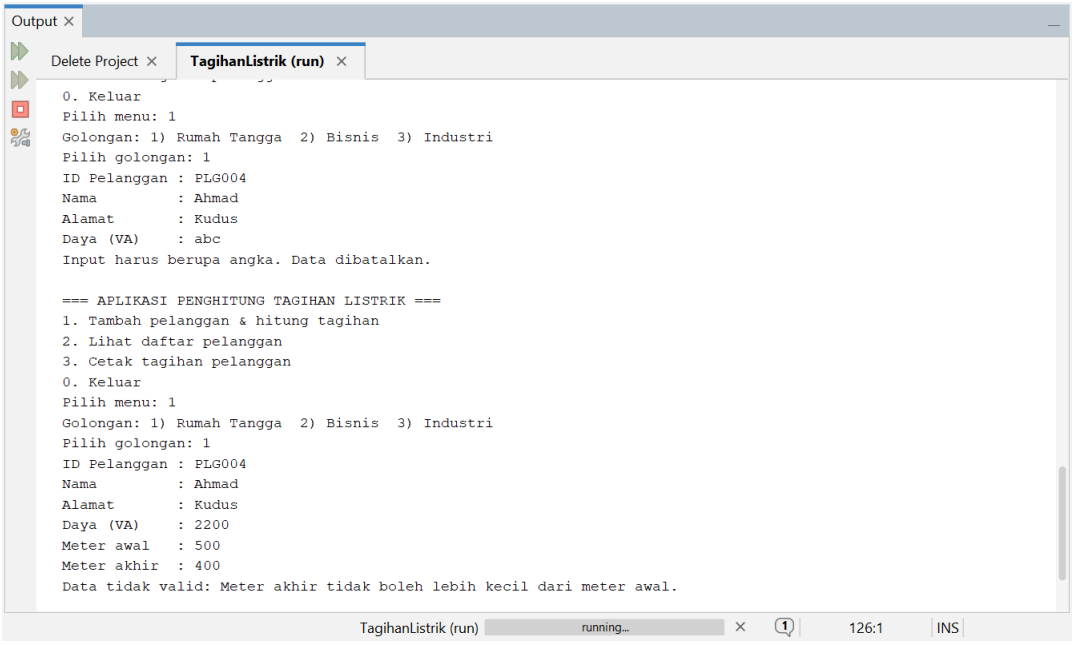
Gambar 6. Tagihan pelanggan Bisnis

7.5 Skenario 3: Pelanggan Industri



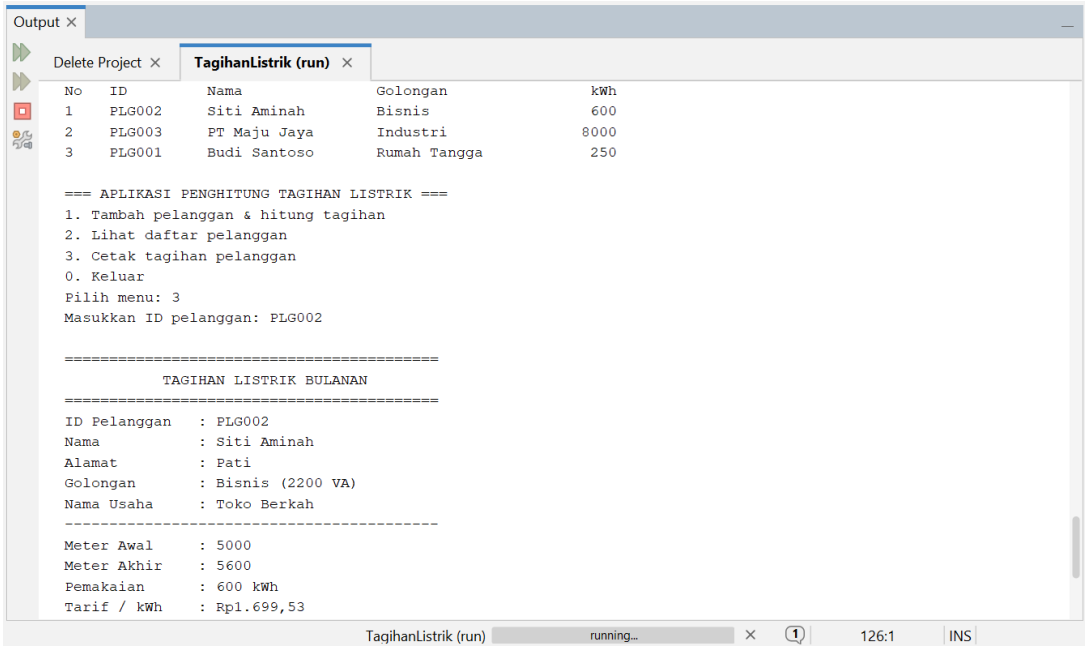
Gambar 7. Tagihan pelanggan Industri

7.6 Skenario 4 dan 5: Validasi Input



Gambar 8. Pesan kesalahan pada validasi input

7.7 Skenario 6 dan 7: Daftar Pelanggan dan Cetak Tagihan



Gambar 9. Daftar pelanggan dan cetak tagihan berdasarkan ID

7.8 Analisis Hasil

Berdasarkan pengujian yang telah dilakukan, Aplikasi Penghitung Tagihan Listrik berjalan dengan baik dan sesuai dengan fungsi yang dirancang. Program berhasil dikompilasi tanpa kesalahan dan dapat dijalankan melalui NetBeans. Seluruh menu yang disediakan, yaitu menambah pelanggan, melihat daftar pelanggan, mencetak tagihan, dan keluar dari program, berfungsi sebagaimana mestinya.

Pada pengujian tiga golongan pelanggan, hasil perhitungan program sesuai dengan hasil perhitungan manual. Pelanggan Rumah Tangga dengan pemakaian 250 kWh menghasilkan total tagihan Rp413.792. Pelanggan Bisnis dengan pemakaian 600 kWh menghasilkan total tagihan Rp1.176.690, dan informasi nama usaha tampil pada tagihan. Pelanggan Industri dengan pemakaian 8.000 kWh menghasilkan total tagihan Rp11.407.462, dengan biaya beban yang dihitung dari daya terpasang. Perbedaan hasil antargolongan menunjukkan bahwa setiap subclass telah menerapkan tarif dan aturan biaya beban masing-masing.

Pengujian validasi juga menunjukkan hasil yang sesuai. Ketika meter akhir diisi lebih kecil dari meter awal, program menolak data dan menampilkan pesan kesalahan. Ketika data numerik diisi dengan huruf, program membatalkan input dan menampilkan pesan bahwa input harus berupa angka. Hal ini membuktikan bahwa enkapsulasi melalui setter dengan validasi dapat menjaga data agar tetap benar dan program tidak berhenti secara tiba-tiba.

Dari sisi penerapan konsep, prinsip *access modifier / encapsulation* terlihat dari atribut yang bersifat private dan hanya dapat diakses melalui getter dan setter. Prinsip *inheritance* terlihat dari tiga subclass yang mewarisi class Pelanggan sehingga tidak perlu menulis ulang atribut dan alur perhitungan yang sama. Selain itu, seluruh objek dapat diproses dengan cara yang sama pada class Main, tetapi hasilnya menyesuaikan jenis pelanggannya.

Secara keseluruhan, tidak ditemukan kendala berarti selama pengujian. Aplikasi ini telah memenuhi tujuan pembuatannya, yaitu menghitung tagihan listrik sekaligus menerapkan konsep *access modifier / encapsulation* dan *inheritance* pada pemrograman berorientasi objek.

7.9 Repository Github

Seluruh kode program dan dokumentasi proyek Aplikasi Penghitung Tagihan Listrik telah disimpan pada repository GitHub agar mudah diakses, diperiksa, dan dikembangkan lebih lanjut. Repository tersebut berisi kode sumber Java pada package tagihanlistrik, yaitu Pelanggan.java, PelangganRumahTangga.java, PelangganBisnis.java, PelangganIndustri.java, dan Main.java, serta gambar diagram class dan laporan tugas ini.

Repository dapat diakses melalui tautan berikut:

https://github.com/cqotada/UTS-PBO-TagihanListrik

Dengan adanya repository ini, dosen maupun pihak lain dapat mengunduh (*clone*) proyek, menjalankannya di NetBeans, dan melihat riwayat perubahan kode selama proses pengerjaan.

8. Kesimpulan

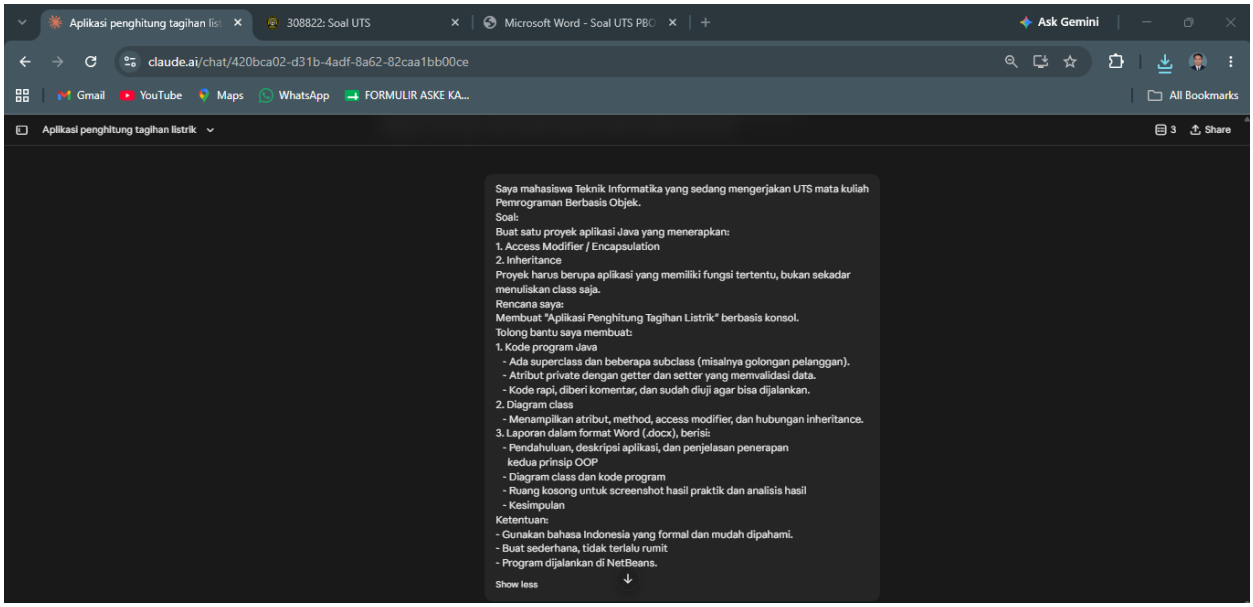
Aplikasi Penghitung Tagihan Listrik berhasil dibuat sebagai aplikasi Java yang berfungsi menghitung tagihan untuk tiga golongan pelanggan. Prinsip enkapsulasi diterapkan melalui atribut private, getter dan setter dengan validasi, serta method protected untuk bagian yang hanya dibutuhkan class turunan. Prinsip inheritance diterapkan melalui superclass abstrak Pelanggan dan tiga subclass yang mewarisinya, sehingga kode tidak berulang dan mudah dikembangkan bila ada golongan pelanggan baru.

Pengembangan lanjutan yang mungkin dilakukan antara lain menyimpan data ke file atau basis data, menambah tarif progresif bertingkat, dan membuat antarmuka grafis.

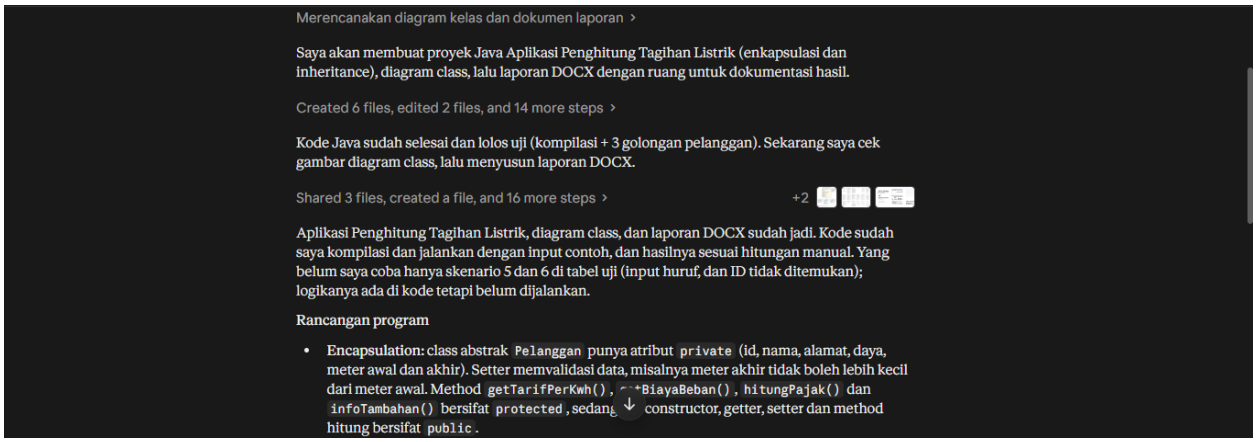
9. Lampiran

Bantuan AI - Ujian Tengah Semester Pemrograman Berbasis Objek
Penggunaan AI : Claude ai (satu kali command)

Screenshot command :



Screenshot Jawaban :



- **Inheritance:** `PelangganRumahTangga`, `PelangganBisnis` dan `PelangganIndustri` mewarisi `Pelanggan` dengan `extends`. `PelangganBisnis` menambah atribut `namaUsaha`, dan `PelangganIndustri` menghitung biaya beban dari daya terpasang.
- **Cara kerja:** `Main` berupa menu konsol untuk menambah pelanggan, melihat daftar, dan mencetak tagihan. Rumusnya total = pemakaian × tarif + biaya beban + PPJ 10%.

Laporan DOCX (18 halaman)
Isinya sampampai kesimpulan: pendahuluan, deskripsi aplikasi, penjelasan kedua prinsip OOP, diagram class, seluruh kode program, dan cara menjalankan. Bagian 7 adalah ruang dokumentasi: 7 skenario uji beserta hasil yang diharapkan, 6 kotak putus-putus untuk menempel screenshot (struktur proyek dan kompilasi, lalu hasil tiap skenario), dan satu kotak kosong untuk analisis hasil. Sampul punya isian Nama, NIM, Kelas, Dosen, dan nama universitas yang perlu Anda isi.

Sebelum dikumpulkan

- Tarif (Rp1.444,70, Rp1.699,53, Rp1.114,74) dan biaya beban adalah nilai simulasi, bukan tarif resmi PLN. Laporan sudah mencantumkan catatan itu.
- Jalankan sendiri program di laptop Anda untuk mengambil screenshot. Angka hasil yang saya tulis di laporan: Rp413.792 (Rumah Tangga, 250 kWh), Rp1.176.690 (Bisnis, 600 kWh), Rp11.407.462 (Industri, 8.000 kWh). Jika berbeda, periksa input Anda.
- Pelajari kodenya sebelum UTS, terutama kenapa tribut `private` dan method abstract di `Pelanggan`, karena dosen biasanya menanyakan itu.

Perintah menjalankannya dari folder proyek: `javac -d out src/tagihanlistrik/*.java` lalu `java -cp out tagihanlistrik.Main`.

File yang saya kirim: `Laporan_UTS_PBO_Tagihan_Listrik.docx`, `TagihanListrik_src.zip` (kode Java dan gambar diagram), dan `diagram_class.png`.



Laporan uts pbo tagihan listrik
Document · DOCX

Tagihanlistrik src
ZIP

Download all

Reply

Claude is AI and can make mistakes. Sonnet 5.5 Medium Manual